Choosing and Validating a Training Method

A decision guide for security problems where someone has already proposed machine learning

Sai Kuruguntla | AIML-501 Professional Portfolio, Artifact 5 | Expanded from an AI-coached session on machine learning training methods (Workshop 3.3)

“Let's use machine learning for this” is a proposal, not a plan. The first decision is not which model to reach for. It is which kind of learning the problem can actually support — and that is fixed by the feedback the environment gives you, not by preference or by what a vendor is selling. This guide is the three-step version I use when a machine learning approach lands on my desk for review.

Step 1 — Identify the feedback the environment gives you

Everything else follows from this. There are three answers, and each one carries a disqualifying condition that matters more than the condition recommending it.

METHOD	FEEDBACK SIGNAL	SECURITY EXAMPLE	DISQUALIFIED WHEN
Supervised	The correct answer is supplied with every example.	Labeling documents PHI or Non-PHI so access controls apply automatically.	No one can produce consistent labels, or the labels encode one reviewer's judgment.
Unsupervised	None. Structure in the data is the only thing to learn from.	Baselining authentication and flow behavior to surface accounts that stop resembling themselves.	You have to defend one specific finding. It can mark a point unusual; it cannot say why.
Reinforcement	Delayed and cumulative — scored on the sequence, not the step.	Learning a containment policy — isolate, throttle, or wait — across a multi-step intrusion.	It cannot be simulated, and production is the only environment you have.

A note on the third row. Reinforcement learning is the one most often proposed and least often appropriate in security operations, because of the credit-assignment problem: the reward arrives at the end of the chain, so the agent must infer which of many earlier actions earned it. In incident response, attributing it to the wrong action does not merely slow learning — it trains the wrong reflex, and the reflex outlives the experiment.

Step 2 — Settle how you would know it worked, before anything is trained

The training loop itself is short: define the objective, split the data, fit the model, evaluate on data the model never influenced, then iterate. Three points in that loop are where reviews usually should have stopped and did not.

The algorithm is a constraint on what can be learned, not an implementation detail.

The algorithm determines what kind of pattern the model is able to represent at all, which means it also determines what the model will never see. Asking which algorithm was chosen and why is the fastest way to find out whether anyone considered the shape of the problem before selecting a tool for it.

Repetition is what converts small corrections into a converged model.

A single pass over the data makes only small adjustments. Iterating is what accumulates them. This is also where the failure in Step 3 begins: the same repetition that produces convergence produces memorization if nothing outside the objective is holding the model honest.

Data plays a different role in each method.

In supervised learning the examples supply the ground-truth patterns that drive optimization — they are the answer key. In unsupervised learning there is no answer key, so the data is the structure to be compressed instead. Confusing the two is what produces the third failure below.

Step 3 — Treat the reported number as flattering until three things are ruled out

Overfitting.

The model memorized the examples rather than the pattern behind them — a detector tuned until it flags every sample from one campaign and nothing from the next. **The check:** a held-out test set touched once, at the end. Every additional look leaks it into the design decisions and quietly demotes it to a second validation set.

Validation leakage.

The held-out score is high and the system still fails in production, because something in the split carried the answer across it — captured traffic divided packet by packet rather than by session, so fragments of one incident sit on both sides; or a feature that is only populated after triage closes. **The check:** split by entity and by time, never by row, and for every feature ask what it contained before the event rather than after it.

Degenerate optimum.

The loss is near zero and the result carries no information. Set k in k-means to the number of samples and you get one cluster per point, zero within-cluster variance, and nothing learned; give an autoencoder a wide enough bottleneck and it copies its input. **The check:** constrain the model from outside its own objective — a bottleneck narrower than the input, a denoising task, or a selection criterion such as the elbow, silhouette score, or BIC.

What this means for whoever signs off

> **An internally defined objective can always be satisfied by a solution that performs no compression and no generalization. So the loss alone can never establish that a result is meaningful — and in the unsupervised case it cannot even choose the model's most basic settings, since k-means loss falls monotonically as k grows.**

Supervised learning has ground truth to keep it honest. Unsupervised learning has to be constrained into honesty by design. That distinction is the whole reason a reviewer exists on this kind of project: the question to bring to a model review is not whether the number is high, but what the number could be true of.

Tool disclosure. The source material for this guide is a coached question-and-answer session on machine learning training methods conducted with Claude (Anthropic), which the course instructor authorized in place of the unavailable SchoolAI coach. The answers in that session were my own; the coach's role was to question them. I used Claude again to draft and format this guide and to generate its two diagrams, and I reviewed the result for technical accuracy. The framing — the disqualifying conditions, the security examples, and the decision to organize the whole guide around reviewing someone else's proposal rather than building my own model — is mine.

References. AIML-501 Workshop 3.3, Machine Learning Training Methods coached activity. Standard classification and clustering evaluation concepts: train/validation/test discipline, data leakage, within-cluster variance, elbow and silhouette selection criteria, Bayesian information criterion, and the credit-assignment problem in reinforcement learning. Text and both diagrams are original to this artifact.